

SEED

Panes y Peces — de unas pocas semillas, multitudes

Wake words propias para Home Assistant a partir de un puñado de grabaciones de tu propia voz, multiplicadas en un dataset rico mediante transformación de timbre.

Método libre para la comunidad · Motor **WORLD vocoder** · Entrenado en local sobre **GPU NVIDIA**

Resultado: **frr 0.0155** (detecta ~98,5%) con **0,187** falsos positivos/hora — a partir de solo **180 muestras**

SEED — Wake words propias para Home Assistant con pocas grabaciones

(Panes y Peces — de unas pocas semillas, multitudes)

Prólogo — o por qué acabas entrenando wake words

Tengo un trabajo en el que he conseguido cerrar el círculo: los que están por encima y los que están por debajo no me echan en falta. Así que me dedico a parecer siempre entretenido con cosas que los demás no entienden, para que no se den cuenta de que es por mí, para mí y conmigo.

Esto es una de esas cosas.

Empezó porque quería que mi Home Assistant respondiera a un nombre elegido por mí, con mi propia voz, corriendo en hardware que es mío — sin nube, sin suscripción, sin los servidores de nadie más que los míos. Lo que sigue es el método al que llegué, la máquina en la que lo monté, y cada muro contra el que choqué por el camino (fueron muchos, y más de una vez me dieron ganas de tirar la toalla).

Quizá tú tengas otro motivo. Quizá quieras resucitar la voz de una secretaria cariñosa que te reciba al llegar. Quizá una voz que le dé algo de sentido a un *Trabajo Basura (Office Space)*, o que le ponga cara a tu propio *Teorema Cero* — ese proyecto en el que buscas un sentido que a lo mejor solo existe porque tú se lo das. Da igual cuál sea — el método es el mismo, y no hace falta ser programador de carrera. Si yo pude montarlo a ratos sueltos, tú también puedes.

Eso sí, aviso: por el camino te vas a encontrar sin dónde agarrarte más de una vez. Como Mad Max en mitad del desierto sin una gota de gasolina para seguir el viaje, hay tramos en los que las herramientas se rompen, el disco se llena, la máquina no arranca y no hay de dónde sacar más. Habrá momentos de *El Hundimiento*, de ver que no hay salida. Se tropieza, se cae y se levanta más veces que en *El Sargento de Hierro* — muchas horas de eso. Se sale rascando de donde se puede (yo resucité un ordenador muerto con dos tarjetas de segunda mano).

Pero al final, cuando ya estás por tirarlo todo, aparece ese hilo de luz que Hellboy llevaba tapado con el puro — la solución que estaba delante todo el rato, solo que no la veías. La sección de *trampas* de más abajo es el mapa de todos los sitios donde me quedé tirado, y de cómo aparté el puro en cada uno, para que tú llegues con el depósito lleno.

De dónde sale esto (genealogía del proyecto)

SEED no nació de la nada: es una **derivación de un proyecto principal** más grande. Ese proyecto principal derivó a su vez en **HA.LAB**, montado en un principio para un tema de **hilo musical** (audio distribuido por la casa/local). De ahí, el hilo musical acabó llevando al **controlador**

de Home Assistant Voice — y una vez metido en la voz, apareció la necesidad de una wake word propia. Que es donde entra este método.

Lo cuento porque estas cosas casi nunca salen de un plan limpio: sales buscando repartir música por las habitaciones y acabas entrenando redes neuronales para que un cacharro responda a tu voz. Cada peldaño abrió el siguiente.

Resumen

En vez de grabar cientos de muestras de tu palabra de activación, grabas **~5-9 tomas cortas "actuadas"** (distintos registros emocionales de tu propia voz), y un script multiplica cada una en ~20 timbres (niño / mujer / hombre / anciano x velocidades) usando el **vocoder WORLD** (manipulación independiente de tono y formantes). El resultado son ~100-180 muestras limpias y variadas a partir de unos minutos de grabación. El entrenador microWakeWord las aumenta luego a 50.000 y entrena el modelo.

Idea central: unas pocas *semillas* bien elegidas + expansión automática supera a grabar por fuerza bruta. Menos esfuerzo, mejor resultado.

Resultado medido: con **180 muestras** SEED (9 registros grabados x 20 timbres) el modelo cuantizado final dio **frr = 0.0155 (detecta ~98,5%) con 0,187 falsos positivos por hora** (cutoff 0.98) — igual o mejor que un modelo anterior mío que necesitó **314 grabaciones a mano**. Menos grabación, mismo resultado (o mejor).

Por qué lo hice

Entrené una wake word propia para mi Voice PE de Home Assistant. El primer intento usó **314 grabaciones reales** de mi voz. Funcionó de maravilla, pero grabar 314 veces es un suplicio, y sospechaba que la mayoría eran redundantes.

El método base de microWakeWord es 100% sintético (Piper TTS). Las muestras de voz real son un *añadido* que reduce falsos positivos y ancla el modelo a cómo dices TÚ la palabra. El consenso de la comunidad es que ~30 muestras reales suele bastar. Así que 314 era pasarse.

La pregunta pasó a ser: **¿puedo obtener la variedad de cientos de grabaciones a partir de unas pocas, transformando mi voz en distintos timbres?** Eso es Panes y Peces (de pocas, multitudes).

El concepto: dos ejes

El truco está en separar dos tipos de variación:

Eje B — REGISTROS (los grabas tú, actuando)

Distintos estados emocionales/prosódicos de tu propia voz. Un filtro NO puede inventarlos — la melodía, las pausas y el énfasis salen de tu boca. Elige registros **separados en el espectro energía x tono** para cubrir el mapa sin solaparse:

Registro	Energía	Tono/velocidad	Zona del espectro
Cansado	baja	grave-lento	esquina inferior
Neutro	media	normal	centro
Con prisa	alta	rápido	lateral rápido
Animado	alta	agudo-vivo	esquina superior
Alterado	muy alta	fuerte-agudo	extremo superior

Grabas ~5 (2-3 tomas de cada uno para variación natural → ~10 grabaciones en total).

Eje A — TIMBRES (los genera un script)

Cada registro se multiplica en distintas "gargantas" — niño, mujer, hombre, anciano — más variantes de velocidad. Esto es puro procesado de señal sobre tu grabación.

Timbre	Qué hace	Variantes
Tu voz	3 tonos x 3 velocidades	9
Niño	formante alto, tono moderado	3
Mujer	formante/tono medio-alto	3
Hombre	formante/tono más bajo	3
Anciano	tono bajo + temblor	2
Total		20

La multiplicación

```
~10 grabaciones (registros) x 20 timbres = ~180 muestras limpias
                                     | (augmentation del entrenador)
                                     50.000 muestras finales
```

Dos expansiones a partir de unos minutos de grabación. Esa es toda la idea. De ahí el nombre: de unas pocas semillas, multitudes.

Por qué el vocoder WORLD (y NO un pitch-shift simple)

Mi primera versión usaba `pyrubberband` (tono + tempo). Problema: subir el tono para hacer un "niño" convertía mi voz en una **ardilla** — antinatural, y el modelo aprendería una voz de dibujos.

La solución fue cambiar al **vocoder WORLD** (`pyworld`), que descompone la voz en tres componentes independientes: - **F0** — el tono - **SP** — envolvente espectral = los FORMANTES (el timbre real / tamaño de garganta) - **AP** — aperiodicidad (el componente de "aire")

Al mover los **formantes por separado del tono**, un niño suena a niño (garganta pequeña) en vez de a ti acelerado. Es el mismo principio que los filtros de formantes de los "voice changer" profesionales.

Clave del afinado: para un niño creíble, sube el **warp de formantes**, no el tono. Tono alto solo = ardilla. Formante alto = cuerpo de niño. Esa idea lo arregló todo.

Qué hace la augmentation (y qué NO debes duplicar)

El entrenador microWakeWord aumenta tus muestras limpias hasta 50.000 aplicando reverbs de sala (RIRs), ruido de fondo y escalado de volumen. Esto significa:

- **Distancia y volumen se hacen automáticamente** — NO los añadas en tu script. Si pre-aplicas una reverb "de lejos" y luego el augmenter añade otra, obtienes "lejos de lejos" — un eco irreal que perjudica al modelo.
- **El 50.000 es un objetivo FIJO.** No crece con tus muestras. La fórmula es: $\text{entornos por muestra} = 50.000 / (\text{tus muestras limpias})$ Menos muestras limpias → cada una se recrea en más entornos. Más muestras → más variedad de contenido, menos entornos cada una. Tú controlas el equilibrio entre *variedad de contenido* (tu trabajo) y *variedad de entorno* (trabajo de la máquina).

Regla: tu script se encarga de lo que cambia la VOZ (timbre, carácter). La augmentation se encarga de lo que cambia el ENTORNO (sala, distancia, ruido, volumen). No los cruces.

Además: no cruces los ejes tono/velocidad sobre los registros. Un registro YA es una combinación de tono+velocidad (una toma "cansada" ya es grave+lenta). Forzar un "cansado-rápido-agudo" suena falso.

El montaje: en qué máquina corre esto (o cómo resucité un Alienware del Área 51)

Todo esto corre sobre una máquina que rescaté: un **Alienware R2** que estaba muerto de asco. Por más que intenté hacerlo revivir por las vías normales (ingeniería inversa incluida), no había manera... hasta que le pinché **dos NVIDIA Titan RTX de 24 GB** y lo empujé con un procesador de **40 hilos**. El bicho resucitó. Fue como rematar una pirámide egipcia poniéndole ascensor y aire acondicionado.

La máquina, que llamo **TITAN**, quedó así:

- **Linux Mint** (Cinnamon).
- **2x NVIDIA Titan RTX** (24 GB cada una, en NVLink) — compradas de segunda mano cuando nadie las quería. Para esto van sobradísimas; con una sola GPU decente basta.
- **CPU de 40 hilos** empujando la generación de muestras (que va por CPU, no por GPU — importa más de lo que parece).
- Acceso remoto por **VNC + noVNC** (para trastear desde el navegador de otra máquina).
- **Docker** + `nvidia-container-toolkit` (el entrenador corre en contenedor).
- Un **NVMe de 1 TB aparte** montado por UUID en `/etc/fstab`, solo para los datos del entrenador (los datasets negativos ocupan ~190 GB; ver "trampas").

Ahora bien — nada de esto es obligatorio. Requisito mínimo real: **una GPU NVIDIA con CUDA y ~6 GB de VRAM** (una GTX 1060 sirve). Mi torre resucitada es overkill; la monté así porque pude y porque era divertido. Entrenar necesita GPU; *ejecutar* el modelo terminado solo necesita el ESP32 del Voice PE (un chip de céntimos).

Para pasar ficheros entre mi PC de grabación (Windows, con Audacity) y la TITAN, monté una **carpeta compartida por Samba**: grabo los registros en Windows, los dejo en la compartida, y el script los procesa en la TITAN.

Cómo se monta el sistema de entrenamiento en local (a rasgos)

Todo el entrenamiento corre en tu propia máquina, sin nube. A grandes rasgos, el montaje es este:

1. **Sistema operativo y drivers.** Linux (yo uso Mint). Instalas los drivers NVIDIA propietarios y el **CUDA toolkit**, de modo que la GPU quede disponible para cómputo. Compruebas que `nvidia-smi` ve la tarjeta.
2. **Docker + soporte de GPU.** Instalas **Docker** y el **nvidia-container-toolkit**, que es lo que permite que un contenedor use la GPU del anfitrión. Sin esto, el contenedor no "ve" la tarjeta.
3. **El entrenador en contenedor.** Todo el software de entrenamiento (TensorFlow, el generador de muestras Piper, el augmentador, los scripts) viene empaquetado en una **imagen Docker** — la de microWakeWord Trainer. No instalas ese lío a mano; lanzas el contenedor y ya lo trae dentro. *Fija una versión de imagen que funcione* (a mí la v11 me va; v13/v14 me crasheaban — ver trampas).
4. **Un disco grande para los datos.** El entrenador descarga y procesa un dataset "negativo" enorme (miles de clips de audio para enseñarle al modelo qué NO es tu palabra). Eso ocupa bastante (~190 GB en mi caso). Montas un disco aparte y apuntas ahí el volumen de datos del contenedor, para no llenar el disco del sistema. Móntalo por **UUID** en `/etc/fstab` (el nombre `/dev/sdX` o `/dev/nvmeX` cambia entre reinicios; el UUID no).

5. **Las voces del generador.** El entrenador crea decenas de miles de muestras sintéticas de tu palabra con voces **Piper TTS**. Para español, descargas las voces `.onnx` (de `HF rhasspy/piper-voices`) y las dejas en la carpeta de voces del generador. Puedes poner varias para más diversidad de hablantes.
6. **Tus muestras personales.** Las muestras que genera tu script (los registros x timbres) se copian a la carpeta `personal_samples/` dentro del contenedor. Son el "toque real" que se suma a las sintéticas.
7. **Interfaz.** El entrenador trae una **web** (un puerto local) donde eliges palabra, idioma y arrancas. También se puede lanzar por línea de comandos (a veces es más fiable — a mí la web se me atascaba). El progreso se sigue por el **log** del contenedor.
8. **El proceso, una vez lanzado**, va solo por fases: **genera** las 50.000 muestras sintéticas → las **augmenta** (les mete reverbs, ruido, distancias) → **entrena** el modelo (aquí es donde la GPU echa humo) → **calibra** y saca el `.tflite` final.
9. **Acceso remoto (opcional).** Como la máquina de entrenar suele estar en un rincón, va bien tener **VNC** (para ver el escritorio) o simplemente **SSH** (para lanzar comandos). Yo uso VNC + noVNC para trastear desde el navegador de otro equipo.

El resultado de todo esto es un fichero `.tflite` de un par de cientos de KB. Ese fichero es lo único que viaja al Voice PE — donde corre sobre un microcontrolador ESP32 sin necesidad de nada de lo anterior.

Paso a paso (lo que hice de verdad)

1. Graba tus registros

En Audacity, en una máquina en silencio, graba ~5 registros (neutro, cansado, prisa, animado, alterado), 2-3 tomas cada uno. Exporta cada uno como WAV. Graba **en silencio** — WORLD es sensible al ruido de fondo. Nómbralos `neutro.wav`, `cansado.wav`, etc.

2. Ejecuta el script de expansión de timbres

Instala dependencias:

```
pip install soundfile pyworld numpy --break-system-packages
```

Ejecuta:

```
python3 panes_y_peces_world.py ./entrada ./salida
```

Lee cada registro y escribe ~20 variantes de timbre por registro (niño, mujer, hombre, anciano x velocidades), normalizadas a 16kHz/16-bit mono. De 9 grabaciones saqué 180 muestras limpias.

3. Cárgalas en el entrenador

Copia las muestras a la carpeta `personal_samples/` del entrenador:

```
sudo docker cp ./salida/. wakeword_trainer:/data/personal_samples/
```

4. Entrena

Pon tu wake phrase, elige idioma (el español funcionó una vez instaladas las voces Piper españolas — ver "trampas"), y arranca. El pipeline: genera 50k muestras TTS → aumenta → entrena → calibra → produce un `.tflite`.

5. Despliega en el Voice PE

Construye un manifiesto JSON de microWakeWord correcto (el `detection_calibration.json` del entrenador NO es el manifiesto de HA), pon el `.tflite` + `.json` en `config/www/wake_words/`, referéncialo en el YAML de ESPHome, compila, flashea. Convive con las wake words de fábrica.

El `.tflite`: cómo sale y cómo hay que dejarlo para que encaje

Esta es una de las partes que más confunde, así que la detallo.

Cómo SALE del entrenador. Al terminar, el entrenador deja en su carpeta de resultados dos ficheros: - El modelo: un `.tflite` (llamado algo como `stream_state_internal_quant.tflite`), de un par de cientos de KB. - Un `detection_calibration.json` con las métricas de calibración (recall/faph por cutoff).

Trampa gorda: ese `detection_calibration.json` **NO es** el manifiesto que Home Assistant / ESPHome espera. Si intentas usarlo tal cual, no funciona. Son cosas distintas: uno son métricas, el otro es la ficha de configuración del modelo.

Qué hay que preparar (ANTES). Sacas el `.tflite` del contenedor y **te fabricas a mano** un manifiesto JSON aparte, con los campos que microWakeWord espera. En esencia lleva: - `type`: `"micro"` - el nombre/ `wake_word` de tu palabra - `model`: la ruta a tu `.tflite` — **con el prefijo `./`** (esto es clave, ver trampas: sin el `./` ESPHome lo rechaza) - `version`: 2 - un bloque `micro` con: `probability_cutoff` (yo uso 0.98-0.99, cuanto más alto más selectivo), `sliding_window_size` (p. ej. 4), `feature_step_size` (10), `tensor_arena_size` (reserva de memoria; ver abajo) y la versión mínima de ESPHome.

Renombra tu `.tflite` a algo propio (p. ej. `mi_wakeword.tflite`) y que el `model` del manifiesto apunte a ese nombre exacto. El `.json` y el `.tflite` van **juntos en la misma carpeta**.

El `tensor_arena_size`. Es la RAM que el modelo reserva en el ESP32. Puedes poner un valor generoso (a mí me sobraba de largo: reservé ~45 KB y el modelo real usaba ~16 KB). Si al compilar sale "Could not allocate tensor arena", súbelo.

Qué pasa DESPUÉS (al flashear). Un par de cosas que te van a pasar: - Tras flashear, el selector de wake word del aparato se queda **vacío** ("No wake word") — hay que volver a elegir la tuya a mano. - Tu palabra **convive** con las de fábrica (Okay Nabu, Hey Jarvis...) — aparecen todas en el desplegable. Si quieres solo la tuya, es cuestión de seleccionar/quitar en la config.

Compilar el firmware: pros, contras y el camino más corto

Para que tu wake word entre en el aparato, hay que **recompilar el firmware del Voice PE** e instalarlo. Hay varias vías, y elegir la buena te ahorra horas:

Vía A — Compilar en el propio Home Assistant (add-on ESPHome). - *Pro:* es lo más integrado; si tu HA tiene RAM de sobra, compila y flashea por OTA sin salir de la interfaz. - *Contra:* si tu HA es modesto (un Yellow con CM4, una Raspberry...), el compilador se queda **sin memoria** y muere (`cc1plus: Killed`). Se puede forzar (parando add-ons, un proceso a la vez), pero es sufrir.

Vía B — Compilar en un Docker de ESPHome en otra máquina con más RAM. - *Pro:* sin ahogos de memoria; la máquina potente hace el trabajo pesado. Reutilizas la misma máquina que usas para entrenar. - *Contra:* hay que montar el contenedor y **fijar la versión de ESPHome** para que case con la de tu HA (si no, se queja de incompatibilidad de config). Un poco más de preparación inicial.

Vía C — Firmware de fábrica + subir solo el modelo. - No siempre aplica, pero si solo cambias el modelo y no tocas el resto de la config, a veces basta con dejar el `.tflite` + `.json` servidos y referenciarlos, sin recompilar todo desde cero.

El camino más corto (lo que recomiendo): 1. Si tu HA tiene RAM holgada → **Vía A**, y a correr. 2. Si tu HA es modesto → **no pelees con su RAM:** monta la **Vía B** (Docker ESPHome en la máquina de entrenar) desde el principio. Lo que pierdes montándolo lo ganas en no chocar contra el `cc1plus: Killed` una y otra vez. 3. Ten preparado el **firmware de fábrica** (`.factory.bin`) para restaurar por USB si un flasheo sale mal — es tu red de seguridad.

(Montar la compilación ESP32 en Docker de forma limpia bien podría ser un mini-proyecto aparte — ver la genealogía; estas cosas siempre derivan en la siguiente.)

Las trampas (la parte honesta — aquí se fueron los días)

Estos son los baches que me costaron tiempo de verdad, y más de una vez me hicieron pensar en dejarlo. Si los pisas, no lo estás haciendo mal — las herramientas tienen aristas.

Bug de la voz TTS española

Con `language=Spanish`, el entrenador crashea al descargar la voz española (`Request.__init__() got unexpected keyword 'headers'` — un descargador roto). **Solución:** descarga a mano voces `.onnx` españolas de HF `rhasspy/piper-voices` y ponlas en `piper-`

`sample-generator/voices/`. El entrenador las encuentra y nunca toca el descargador roto. (Puedes poner varias — `es_ES`, `es_AR`, `es_MX` — y usa todas las que coincidan con `es_*.onnx`, más diversidad de hablantes.)

El disco se llena (dos veces)

La preparación de datasets negativos (resample FMA→WAV de 8000 ficheros) me llenó el SSD raíz **dos veces** con los flags de limpieza desactivados (guarda ZIP + descomprimido + WAV). Apunta el volumen de datos del entrenador a un disco grande. Móntalo por **UUID** en fstab — el nombre `/dev/nvmeX` baila entre reinicios.

Versiones de la imagen del entrenador

La imagen `v11` funciona. `v13` / `v14` crashean al arrancar con un bug de `dirname` / `NoneType`. Fija la v11.

El generador se cuelga en 50000/50000 (con voces ONNX)

El proceso wrapper de progreso gira eternamente (busy-wait sobre un hijo muerto = zombie) tras generar la última muestra, sin pasar nunca a la augmentation. **Rescate:** mata el wrapper, escribe la etiqueta esperada en `work/last_wake_word` (`wakeword: 50000:es+voz1.onnx+voz2.onnx+...`), y relanza — el entrenador ve que las muestras ya existen ("Sample generation not required") y salta directo a la augmentation. Ahorra ~80 min de regeneración. (Me pasó dos veces; el rescate funciona las dos.)

RAM para compilar el firmware (el HA Yellow CM4)

El firmware del Voice PE lo compilé en un **Home Assistant Yellow con Compute Module 4 (CM4)** — que es una máquina modesta de RAM. La compilación murió con `cc1plus: Killed` (sin memoria), siempre en los ficheros de `esp-tflite-micro`. Al final **Sí pudo compilar bien en el propio Yellow**, pero a base de **matar procesos y liberar recursos temporales**: parando todos los add-ons para dejar RAM libre y limitando la compilación a un solo proceso a la vez. Ninja retoma donde quedó — no hagas "clean build files", o empieza de cero.

Ahora bien, hacerlo así es sufrir. **Lo ideal es no depender de la RAM del Yellow**: montar la compilación en un **Docker de ESPHome en otra máquina con más memoria** (por ejemplo la misma que usas para entrenar). Fijas la versión de la imagen ESPHome para que case con la de tu HA, le das el YAML del dispositivo + los ficheros de wake word, y compilas ahí sin ahogos. Eso podría ser un paso aparte del proyecto — montar el compilador de firmware ESP32 en contenedor, separado del entrenador.

La ruta del manifiesto en ESPHome

El campo `"model"` del manifiesto JSON necesita el prefijo `./` (`"model": "./mimodelo.tflite"`) o ESPHome lo rechaza como "not a valid model name, local path, http(s) url". Las URLs HTTP también fallan en ESPHome actual — usa ruta relativa con el `.json` y el `.tflite` juntos.

El selector de wake word se resetea tras flashear

Tras flashear, el selector del Voice PE queda en "No wake word" (vacío) en vez de tu palabra propia — ponlo a mano o el aparato no escucha nada.

WORLD odia las grabaciones rápidas/ruidosas

Mi toma "con prisa" se generó incompleta — la detección de F0 de WORLD sufre con voz muy rápida o ruidosa. Graba una segunda toma, o graba la de prisa un poco más limpia.

Pros y contras (mi opinión honesta)

Pros

- **Muchísima menos grabación.** 9 tomas vs 314. Minutos en vez de una tarde de sufrimiento.
- **Mucha variedad con poco esfuerzo.** Cada registro se vuelve 20 timbres; el augmenter hace 50k. Doble expansión desde una semilla.
- **WORLD da voces naturales** — niño/mujer/anciano creíbles, no ardilla.
- **Conservas TU pronunciación.** A diferencia del TTS puro (que cambia toda la voz), los timbres mantienen tu prosodia y acento — el modelo sigue aprendiendo cómo lo dices tú.
- **Reproducible y editable.** La tabla de timbres son solo números; añades una voz añadiendo una fila.

Contras / advertencias

- **Necesita GPU NVIDIA** para entrenar (ejecutar el modelo no necesita nada — corre en el ESP32).
 - **WORLD es más lento** que el pitch-shift simple y sensible al ruido de grabación.
 - **El entrenador tiene aristas** (ver trampas). Cuenta con pelearte con las herramientas.
 - **Los timbres extremos son contraproducentes.** Voces de demonio/ogro/robot aumentan los falsos positivos — el modelo aprende a activarse con voces que no son la tuya. Mantén los timbres en rango humano.
-

Ficheros

(*Rellena tus enlaces*) - `panes_y_peces_world.py` — el script de expansión de timbres - Grabaciones de registros de ejemplo - El `.tflite` entrenado + manifiesto (como referencia)

Créditos

- Construido sobre el [entrenador microWakeWord](#) de TaterTotterson

- Vocoder WORLD vía [pyworld](#)
- Voces Piper de [rhasspy/piper-voices](#)

Método y script compartidos libremente para la comunidad. Usa, adapta, mejora.